



河南工业大学

StudyMate：基于大模型的
个性化资源生成与学习多智能体系统

项目部署文档

作品名称： StudyMate：基于大模型的个性化资源生成与学习多智能体系统

申报者姓名： -----

集体申报全体成员姓名： -----、-----、-----、-----、-----

目 录

1. 项目概述	1
1.1 项目简介	1
1.2 技术栈	1
1.3 架构图	2
2. 系统要求	2
2.1 硬件要求	2
2.2 软件要求	2
2.3 网络要求	2
3. 部署前准备	3
3.1 服务器信息收集	3
3.2 本地准备	3
3.3 服务器准备	3
4. 部署架构说明	4
4.1 整体部署架构	4
4.2 各组件职责说明	5
4.3 部署流程概述	6
4.4 部署注意事项	7
5. 详细部署步骤	8
5.1 环境准备	8
5.2 项目配置	12
5.3 Piston 镜像与运行时	16
5.4 Docker Compose 服务配置	18
5.5 一键启动与更新	24
5.6 权限与安全加固	28
6. SSL 证书配置	29
6.1 Caddy 自动 HTTPS	29
6.2 证书验证与续期	30
7. 验证部署	31
7.1 检查服务状态	31
7.2 检查端口与公网访问	32
7.3 核心功能验收	33
7.4 数据与代码运行验收	34
7.5 运行监控与验收结论	35
8. 常见问题排查	36
8.1 容器、反向代理与镜像问题	36
8.2 HTTPS、Piston 与前端缓存问题	37
9. 日常运维	38
9.1 查看日志与服务管理	38
9.2 数据备份与恢复	39
9.3 性能与资源管理	39
10. 附录	40
10.1 文件说明与部署状态	40
10.2 测试网址与账号	41
10.3 项目代码与交付清单	42

1. 项目概述

1.1 项目简介

StudyMate 是一套面向高校计算机类课程的个性化资源生成与学习多智能体系统。系统围绕“学习画像—知识检索—资源生成—练习评估—画像更新”构建闭环，将课程知识、学习目标、智能问答、笔记、测验、学习报告和就业能力建议连接起来。

当前平台覆盖机器学习、数据结构与算法、操作系统、计算机网络、计算机组成原理五门课程，支持课程级知识库隔离、RAG 原文追溯、多轮 AI 助教、语音辅导、可视化动画以及在线代码运行。

个性化学习方面，系统维护知识基础、认知风格、学习目标、薄弱点、学习节奏、内容偏好和就业技能 7 组动态画像，并在生成讲解、练习、代码案例和复习路径时自动带入课程上下文，使相同知识点能够按照不同学习者的掌握程度进行表达。

资源生成方面，多个智能体分别承担知识检索、讲解文档、思维导图、智能测验、代码示例、拓展阅读和学习路径规划任务；生成结果统一保存到工作台，便于继续编辑、复习、导出和追溯来源。

交互与评估方面，AI 助教支持 SSE 流式回复、图片和文档附件，在线编程模块支持 Python、C11 与 C++17；笔记、错题、测验和学习报告共同记录学习过程。系统还接入哔哩哔哩与人才呀公开学习目录，并将论文、图书和博客标题解析为经过白名单与标题校验的真实详情页；岗位建议仍在本地根据画像、课程和测验历史生成。

1.2 技术栈

类别	技术
前端	React 19、TypeScript、Vite 8、Tailwind CSS 4、Nginx
后端	Python 3.11、FastAPI 0.115、Uvicorn、SQLAlchemy 2.0
智能体	LangGraph、OpenAI 兼容 SDK、SSE 流式输出
数据与检索	SQLite、BM25、Qwen Embedding、RRF 混合检索
公网入口	Caddy 2、自动 HTTPS、zstd/gzip
代码沙箱	Piston、Python 3.10、GCC 10.2 (C11/C++17)
部署	Docker Engine、Docker Compose、Named Volume

1.3 架构图

StudyMate 生产部署架构

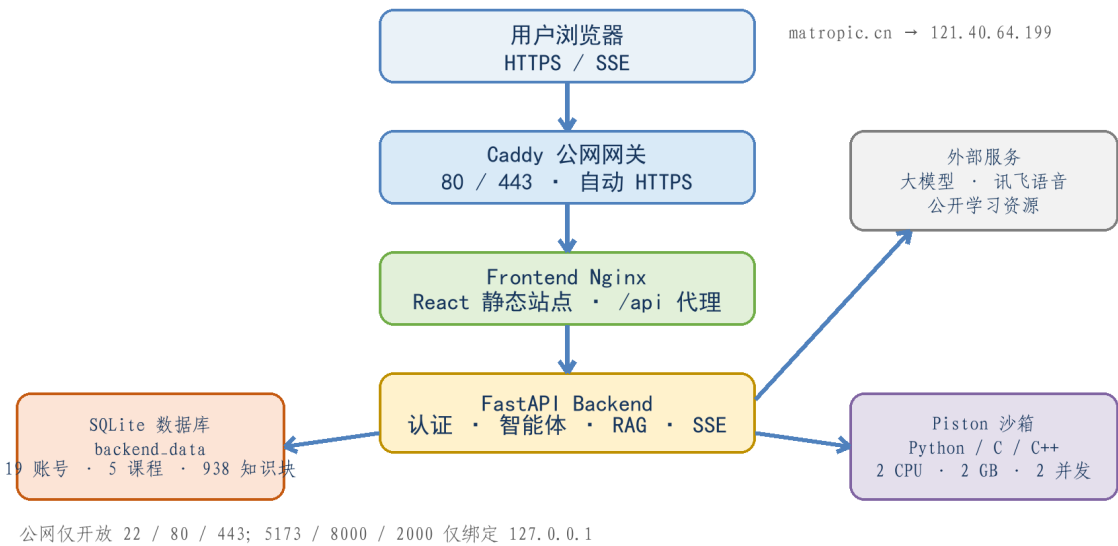


图 1-1 StudyMate 基本部署架构图

2. 系统要求

2.1 硬件要求

项目	最低配置	推荐配置	当前服务器
CPU	2 核	4 核及以上	4 vCPU
内存	4 GB	8 GB 及以上	7.1 GiB
磁盘	30 GB	60 GB 及以上	59 GB (约 46 GB 可用)
带宽	5 Mbps	10 Mbps 及以上	满足竞赛演示

2.2 软件要求

Ubuntu 22.04 LTS 64 位; Docker Engine 24+; Docker Compose V2+; SSH、Rsync、Curl。当前生产环境为 Ubuntu 22.04.5、Docker 29.6.1、Compose v5.3.1。

2.3 网络要求

服务器需要公网 IPv4、已备案域名及可用 DNS。公网只开放 22、80、443; 2000、5173、8000 等应用端口仅绑定 127.0.0.1。

3. 部署前准备

3.1 服务器信息收集

信息项	本项目配置
公网 IP	121.40.64.199
操作系统	Ubuntu 22.04.5 LTS 64 位
SSH 用户	deploy（普通部署用户）
SSH 别名	studymate-server
域名	matropic.cn
备案号	豫ICP备2026028221号
部署目录	/home/deploy/studymate

3.2 本地准备

确认前后端生产构建通过，并重新生成脱敏压缩 SQLite 种子库。
准备 backend/.env，但不得把真实密钥写入文档或提交到代码仓库。
确认域名 A 记录指向服务器公网 IP，备案信息与页面展示一致。

上传前应删除或排除 node_modules、dist、.env、日志、本地运行库和历史备份，同时确认 backend/resources/seed/studymate.db.gz 仍在项目中。该脱敏压缩文件是首次启动的数据源，包含固定测试账号、五门课程和 938 条课程知识块。

3.3 服务器准备

```
ssh studymate-server
uname -a
df -h /
ss -lntp
```

检查结果应确认系统为 Ubuntu 22.04 x86_64、磁盘空间充足，且 80、443 端口没有被旧版 Nginx、Apache 或其他网关占用。服务器上的日常部署目录统一使用 /home/deploy/studymate。

注意事项：部署用户没有 sudo 权限时，应在云厂商“云助手”中以 root 身份运行一次初始化脚本，不应索要或传播 root 密码。

4. 部署架构说明

4.1 整体部署架构

生产环境采用 Docker Compose 单机容器化架构。Caddy 是唯一公网入口，负责 80/443、TLS 证书和压缩；前端 Nginx 托管 React 静态资源并代理 /api；FastAPI 提供认证、课程、RAG、SSE、上传与代码运行接口。

StudyMate 生产部署架构

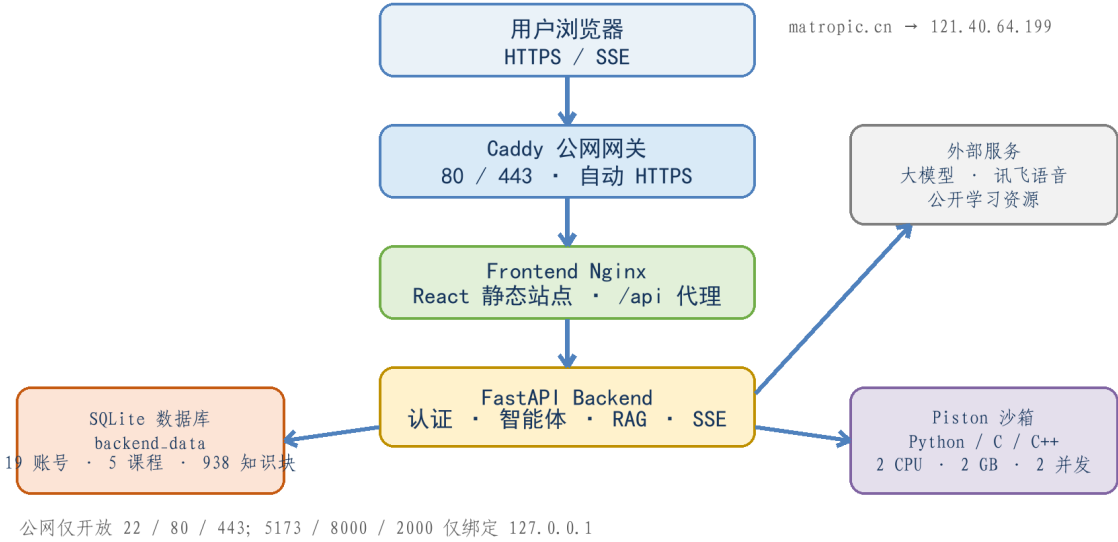


图 4-1 项目整体部署架构图

网络层次	访问范围
公网	Caddy: 80/443
宿主机回环	frontend 5173、backend 8000、Piston 2000
Docker 内网	192.168.242.0/24，服务名互访
外部服务	大模型、语音、邮件、哔哩哔哩、人才呀及公开阅读目录

4.2 各组件职责说明

四个核心容器通过同一 Compose 网络互联，外部请求只允许进入 Caddy。前端、后端和代码沙箱保留宿主机的回环端口用于排障，但不在云安全组中开放，从网络边界上减少直接攻击面。

4.2.1 Caddy—公网网关

自动申请和续期 Let's Encrypt 证书。
HTTP 自动跳转 HTTPS。
开启 zstd/gzip, SSE 使用低延迟转发。

4.2.2 Frontend Nginx—静态站点

托管 React/Vite 构建产物。
支持 SPA 路由回退。
将 /api 转发到 backend:8000, 上传限制 20MB。

4.2.3 FastAPI—业务服务

用户、角色与安全会话。
课程、知识库、7 组画像、笔记、测验、报告和就业建议。
SSE 流式 AI、文件解析、语音、外部公开资源、可信阅读直链解析和代码执行转发。

4.2.4 SQLite 与 Named Volume

线上数据库位于 backend_data 卷，容器重建不会覆盖；Piston runtime 和 Caddy 证书也分别持久化。

4.2.5 Piston—在线代码沙箱

支持 Python、C、C++；容器整体限制为 2 CPU、2GB、256 PID、2 并发，避免共享测试账号耗尽宿主机资源。

上述组件均设置 restart: unless-stopped。宿主机重启后 Docker 服务会自动拉起容器；数据库、语言运行时和证书保存在命名卷中，镜像更新不会改变已有业务数据。

4.3 部署流程概述

StudyMate 部署流程



图 4-2 StudyMate 部署流程

阶段	主要工作	参考耗时
阶段 1: 基础环境	检查服务器、安装 Docker、配置镜像源与防火墙	10 ~ 20 分钟
阶段 2: 项目与配置	Rsync 上传、单独上传环境变量、校验数据	5 ~ 10 分钟
阶段 3: 镜像与运行时	构建前后端、导入 Piston、初始化 Python/GCC	10 ~ 30 分钟
阶段 4: 公网入口	启动 Caddy、签发 HTTPS 证书	1 ~ 5 分钟
阶段 5: 验收	登录、AI、上传、代码运行、数据库完整性	10 分钟

首次部署受镜像和 Piston runtime 下载速度影响；后续更新可复用构建缓存和持久化卷，通常只需数十秒至数分钟。

4.4 部署注意事项

权限：仅一次性系统初始化需要 root；日常 Compose 操作使用 deploy 用户。
网络：云安全组和 UFW 必须同时开放 80/443，DNS A 记录必须先指向服务器。
密钥：backend/.env 权限为 600，Rsync 时必须排除，禁止写入文档和镜像。
数据：backend_data 是线上数据库，不会因 --build 自动覆盖。
镜像：Docker Hub 走国内加速；GHCN 的 Piston 必要时使用导入脚本。
端口：2000、5173、8000 不对公网开放。
更新：允许 docker compose down，但严禁带 -v。
代码沙箱：Piston 使用 privileged 运行 isolate，因此必须保留资源上限。

数据保护红线：严禁执行 docker compose down -v。该命令会删除数据库、Piston runtime 和 Caddy 证书等命名卷。

建议所有变更遵循“备份—同步—构建—健康检查—业务验收”的顺序。

比赛展示前应至少完成一次从公网网络发起的全流程验收，并保留最近一份可恢复数据库备份。变更域名、Cookie、安全组或 Caddyfile 后，必须重新检查 HTTPS、登录状态和 SSE 流式响应。

4.4.1 变更前检查清单

变更对象	需要保留	完成后验证
前端源码	backend_data、生产 env	首页、静态资源、/api
后端源码	数据库卷、密钥	/api/ping、登录、AI SSE
Compose	四个命名卷	服务、网络、端口映射
Caddyfile	caddy_data/config	证书、HTTP 跳转、SSE
Piston	piston_data	runtime、Python/C++
数据库	当前备份与回滚点	integrity/FK/核心数量

5. 详细部署步骤

5.1 环境准备

5.1.1 检查服务器

```
ssh studymate-server
cat /etc/os-release
uname -m
nproc
free -h
df -h /
ss -lntp
```

预期结果：Ubuntu 22.04、x86-64、磁盘剩余空间不少于 15GB，80/443 没有被其他 Web 服务占用。

5.1.2 上传初始化脚本

```
scp studymate/scripts/bootstrap-ubuntu.sh \
    studymate-server:/home/deploy/studymate-bootstrap.sh
ssh studymate-server 'chmod 755 ~/studymate-bootstrap.sh'
```

注意事项：若 deploy 不在 sudoers 中，请使用阿里云 ECS 云助手，以 root 身份执行下一页的命令。

5.1.1.1 时间、DNS 与出站连通性

```
timedatectl status
getent ahostsv4 matropic.cn
curl -I --max-time 10 https://mirrors.aliyun.com
curl -I --max-time 10 https://api.deepseek.com

# 预期：系统时间同步、域名解析正确、HTTPS 出站可达
```

系统时间偏差会导致 TLS 和 Cookie 验证异常；DNS 或 HTTPS 出站受限会同时影响镜像下载、证书签发和模型调用，应在安装软件前排除。

5.1.3 安装 Docker Engine 与 Compose

推荐使用项目自带的 Ubuntu 初始化脚本。脚本会自动选择可访问的 Docker CE 软件源，并安装 Docker、Compose plugin、Skopeo 和 UFW。

```
# 在云助手中以 root 身份执行
DEPLOY_USER=deploy \
  bash /home/deploy/studymate-bootstrap.sh \
  2>&1 | tee /home/deploy/studymate-bootstrap.log
```

看到 Bootstrap complete 后，退出并重新建立 SSH 会话，使 deploy 用户获得 docker 组权限。

```
ssh studymate-server
id -Gn
docker --version
docker compose version
docker info --format 'server={{.ServerVersion}}'
```

验证项	实际验收结果
Docker	29.6.1
Docker Compose	v5.3.1
用户组	deploy docker
服务状态	docker active

5.1.3.1 安装结果复核

```
systemctl is-active docker
systemctl is-enabled docker
docker version --format '{{.Server.Version}}'
docker compose version
docker buildx version
skopeo --version
id deploy
```

Docker 应为 active/enabled，deploy 的 groups 中应包含 docker。若当前会话仍提示 docker.sock 权限不足，退出 SSH 后重新连接，不要把 socket 权限改成 777。

5.1.4 配置中国大陆镜像源

初始化脚本将 Docker CE 软件源按“阿里云—腾讯云—中科大—官方源”自动回退，并写入 Docker Hub 镜像加速配置。

```
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io",
    "https://docker.lms.run",
    "https://dockerproxy.net"
  ],
  "log-driver": "local",
  "log-opts": {"max-size": "100m", "max-file": "5"}
}
```

```
sudo dockerd --validate \
  --config-file /etc/docker/daemon.json
sudo systemctl restart docker
docker info --format '{{json .RegistryConfig.Mirrors}}'
```

后端镜像构建还会使用阿里云 Debian 源和阿里云 PyPI，前端 npm 使用 npmmirror，避免仅配置 Docker 镜像源后依赖安装仍然卡住。

5.1.4.1 构建依赖源验证

```
docker pull alpine:3.20
docker run --rm alpine:3.20 cat /etc/alpine-release

# 前端构建使用
npm config get registry
# 后端构建参数
echo http://mirrors.aliyun.com/debian
echo https://mirrors.aliyun.com/pypi/simple
```

Docker daemon 镜像加速只处理 Docker Hub；Debian、PyPI、npm 和 GHCR 各自需要独立策略。遇到超时时应根据失败域名定位对应源，而不是反复更换同一项配置。

5.1.5 配置安全组与 UFW

云安全组和 Ubuntu 防火墙都需要放行必要端口。配置 UFW 时先允许 SSH，再启用防火墙，避免远程连接被锁断。

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
sudo ufw status numbered
```

端口	用途	公网策略
22/TCP	SSH 维护	建议仅维护人员 IP
80/TCP	ACME 验证与 HTTPS 跳转	开放
443/TCP	HTTPS	开放
2000/TCP	Piston	禁止开放
5173/TCP	前端排障	禁止开放
8000/TCP	后端排障	禁止开放

注意事项：安全组是云平台层，UFW 是操作系统层；只配置其中一层并不能保证公网可访问。

阿里云安全组入方向建议把 22 端口来源限制为维护人员固定 IP，80、443 允许公网访问；出方向至少允许 DNS、HTTP 和 HTTPS，以便 Docker 拉取镜像、Caddy 申请证书以及后端调用 AI 服务。

```
sudo ufw status verbose
curl -I --max-time 10 https://registry-1.docker.io
curl -I --max-time 10 https://acme-v02.api.letsencrypt.org/directory
```

5.2 项目配置

5.2.1 上传项目文件

```
ssh studymate-server 'mkdir -p ~/studymate'

rsync -az --delete --progress \
  --exclude '.git/' \
  --exclude 'frontend/node_modules/' \
  --exclude 'backend/.venv/' \
  --exclude 'backend/.env' \
  --exclude 'backend/backups/' \
  --exclude 'backend/studymate.db' \
  --exclude 'backups/' \
  --exclude '.deploy.env' \
  --exclude '*.log' \
  ./studymate/ studymate-server:~/studymate/
```

backend/resources/seed/studymate.db.gz 是首次启动所需的脱敏演示种子库，应随项目上传。
backend/studymate.db 是开发机本地运行库，必须排除。生产 backend/.env 和 .deploy.env 必须单独维护。

```
scp studymate/backend/.env \
  studymate-server:~/studymate/backend/.env
ssh studymate-server \
  'chmod 600 ~/studymate/backend/.env'
```

5.2.1.1 上传后核对

```
ssh studymate-server '
  cd ~/studymate
  test -f docker-compose.yml
  test -f Caddyfile
  test -f backend/resources/seed/studymate.db.gz
  test -x scripts/deploy.sh || chmod +x scripts/*.sh
  du -sh .
  find . -maxdepth 2 -type f | sort | sed -n "1,40p"
'
```

核对应确认生产 env 未被普通 Rsync 覆盖，压缩种子库和部署脚本存在，且目录所有者仍为 deploy。
不要把本地 studymate.db、node_modules、.venv 或历史备份上传到服务器。

5.2.2 演示数据准备

公开竞赛环境只保留指定管理员、评委和学生账号，同时保留课程、知识块、画像和必要演示数据。项目通过独立构建脚本从本地运行库生成可提交的脱敏压缩种子，不直接提交本地数据库。

```
cd studymate
python3 scripts/build_seed_db.py
gzip -t backend/resources/seed/studymate.db.gz
```

数据项	验收结果
用户	19（1 管理员、10 评委、8 学生）
课程	5
知识块	938
SQLite integrity-check	ok
外键错误	0

构建脚本使用 SQLite backup API 获取一致快照，只修改快照，不修改 backend/studymate.db；随后删除未批准账户的关联记录、清空会话和一次性验证码、校验外键与完整性，最后写出可复现的 gzip 文件。

```
ls -lh backend/resources/seed/studymate.db.gz
python3 - <<'PY'
import gzip, shutil, sqlite3, tempfile
with tempfile.NamedTemporaryFile(suffix='.db') as f:
    with gzip.open('backend/resources/seed/studymate.db.gz', 'rb') as src:
        shutil.copyfileobj(src, f)
    f.flush()
    c = sqlite3.connect(f.name)
    print(c.execute('PRAGMA integrity_check').fetchone())
    print(c.execute('PRAGMA foreign_key_check').fetchall())
PY
```

注意事项：不要直接用手工 SQL 批量删除用户，也不要将本地运行库改名后当作种子提交。应始终通过 build_seed_db.py 生成部署产物。

5.2.3 配置生产环境变量

```
cd ~/studymate
cp .deploy.env.example .deploy.env
chmod 600 .deploy.env
```

```
COMPOSE_PROFILES=public,code-runner
SITE_ADDRESS=matropic.cn
HTTP_PORT=80
HTTPS_PORT=443

CADDY_IMAGE=docker.m.daocloud.io/library/caddy:2-alpine
PYTHON_IMAGE=docker.m.daocloud.io/library/python:3.11-slim
NODE_IMAGE=docker.m.daocloud.io/library/node:20-alpine
NGINX_IMAGE=docker.m.daocloud.io/library/nginx:alpine
```

backend/.env 中至少应确认以下生产项：

```
CORS_ORIGINS=https://matropic.cn
SESSION_COOKIE_SECURE=true
AUTH_SECRET_KEY=<随机强密钥>
DEEPSEEK_API_KEY=<仅服务器保存>
SPARK_API_KEY=<仅服务器保存>
```

注意事项：文档只展示变量名与占位符。真实 LLM、语音、邮件、认证密钥不得出现在截图、PDF 或代码仓库中。

5.2.3.1 关键变量说明

变量	作用	生产要求
SITE-ADDRESS	Caddy 站点地址	matropic.cn
COMPOSE-PROFILES	启用网关/沙箱	public,code-runner
CORS-ORIGINS	允许浏览器来源	仅正式 HTTPS 域名
SESSION-COOKIE-SECURE	限制 Cookie 传输	true
AUTH-SECRET-KEY	会话签名	随机强密钥，禁止公开
PISTON-URL	后端访问沙箱	Compose 内网服务名
LLM/API Key	模型与语音能力	仅服务器保存

5.2.4 校验 Compose 配置

```
cd ~/studymate
docker compose --env-file .deploy.env config --quiet
docker compose --env-file .deploy.env config --services
```

生产 profile 应包含 backend、frontend、piston-api、caddy 四个核心服务。PostgreSQL、Redis、Chroma 属于 extras 可选 profile，当前生产环境没有启用。

服务	容器名	端口映射	持久化
backend	studymate-backend	127.0.0.1:8000	backend-data
frontend	studymate-frontend	127.0.0.1:5173	镜像内静态文件
piston-api	studymate-piston	127.0.0.1:2000	piston-data
caddy	studymate-caddy	80、443	caddy_data/config

默认 Docker bridge 使用 192.168.242.0/24，服务之间通过 Compose 服务名通信。

5.2.4.1 服务依赖与内部网络

```
# config --services 的生产输出
backend
piston-api
frontend
caddy

# 固定 Docker bridge
subnet: 192.168.242.0/24
gateway: 192.168.242.1

# 服务间使用 backend:8000、piston-api:2000、frontend:80
```

内部地址由 Docker DNS 解析，容器不应通过宿主机公网 IP 互相访问。固定子网便于排障，但业务代码仍应优先使用 Compose 服务名，避免依赖变化的容器 IP。

5.3 Piston 镜像与运行时

5.3.1 导入 Piston 镜像

Piston 镜像位于 GHCR, Docker Hub 镜像加速不能覆盖该仓库。先尝试项目导入脚本:

```
cd ~/studymate
bash scripts/import-piston-image.sh
docker image inspect \
  ghcr.io/engineer-man/piston:latest
```

脚本先调用 Docker 原生拉取; 超时后使用 Skopeo 写入临时 docker-archive, 再由当前 Docker CLI 导入, 从而规避 Ubuntu 22.04 旧版 Skopeo 与 Docker 29 API 不兼容的问题。

5.3.2 离线镜像传输 (备用)

```
docker save ghcr.io/engineer-man/piston:latest \
  | gzip -1 \
  | ssh studymate-server 'gzip -dc | docker load'
```

注意事项: 镜像传输完成后保留原始镜像名, Compose 无需修改。

5.3.1.1 镜像导入校验

```
docker image inspect ghcr.io/engineer-man/piston:latest \
  --format '{{.Id}} {{.Architecture}} {{.Os}}'
docker image ls ghcr.io/engineer-man/piston --digests

# 预期: linux / amd64, 镜像标签保持原名
# Compose 启动后:
docker inspect studymate-piston --format '{{.State.Status}}'
```

导入成功的关键是本地镜像名与 Compose 中的 PISTON_IMAGE 一致。若 docker load 后只有镜像 ID 没有标签, 应重新 docker tag, 而不是修改多处 Compose 配置。

5.3.3 初始化 Python 与 GCC runtime

```
cd ~/studymate
bash scripts/init-piston.sh
curl http://127.0.0.1:2000/api/v2/runtimes
```

验收结果应包含 Python 3.10.0，以及由 GCC 10.2.0 提供的 C、C++、D 和 Fortran runtime。项目主要使用 Python、C11、C++17。

语言	版本	用途
Python	3.10.0	解释运行
C	GCC 10.2.0	-std=c11 -O2
C++	GCC 10.2.0	-std=c++17 -O2

运行时保存在 piston-data 卷，后续容器重建会复用。初始化脚本会跳过已安装包，并在结束时强制校验 Python 与 C++ 是否存在。

```
curl -sS http://127.0.0.1:2000/api/v2/runtimes \
  | python3 -m json.tool

# 预期至少出现 python 3.10.0、c 10.2.0、c++ 10.2.0
```

运行时初始化只需在首次创建 piston-data 或更换 Piston 基础镜像后执行。正常前后端更新不需要重新下载语言包，因此应优先保留命名卷并复用既有 runtime。

注意事项：境外下载过慢时，可从本机已验证的 piston-data 中迁移 runtime；不要反复删除卷重新下载。

5.4 Docker Compose 服务配置

5.4.1 一键部署脚本

```
#!/usr/bin/env bash
set -euo pipefail

docker compose --env-file .deploy.env \
  up -d --build --remove-orphans

bash scripts/init-piston.sh
docker compose --env-file .deploy.env ps
```

实际 `scripts/deploy.sh` 会自动识别 `.deploy.env`，支持 `up`、`status`、`logs`、`down` 四种操作。`up` 使用 `--build` 构建前后端，等待健康检查后启动 Caddy，并幂等初始化 Piston。

```
bash scripts/deploy.sh
bash scripts/deploy.sh status
bash scripts/deploy.sh logs
bash scripts/deploy.sh down
```

`down` 特意不带 `-v`，因此只停止并删除容器和网络，不删除业务卷。

5.4.1.1 `deploy.sh` 操作分支

```
case "${1:-up}" in
  up)
    docker compose --env-file .deploy.env \
      up -d --build --remove-orphans
    bash scripts/init-piston.sh
    docker compose --env-file .deploy.env ps ;;
  status) docker compose --env-file .deploy.env ps ;;
  logs)   docker compose --env-file .deploy.env \
    logs -f --tail=200 backend frontend caddy piston-api ;;
  down)   docker compose --env-file .deploy.env down ;;
  esac
```

该脚本不包含 `down -v`、`system prune` 或数据库覆盖操作，可作为日常更新入口。遇到构建失败时原有容器通常仍可继续运行，应先读取失败步骤再决定回滚。

5.4.2 前端镜像构建

前端使用多阶段 Dockerfile: Node 20 安装依赖并执行生产构建, Nginx Alpine 仅保留 dist 静态文件。npm 使用 npmmirror。

```
ARG NODE_IMAGE=node:20-alpine
ARG NGINX_IMAGE=nginx:alpine
FROM ${NODE_IMAGE} AS build
WORKDIR /app
RUN npm config set registry https://registry.npmmirror.com
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM ${NGINX_IMAGE} AS runtime
COPY --from=build /app/dist /usr/share/nginx/html
```

生产构建已验证 TypeScript、Vite、备案页脚和哈希静态资源。Nginx 健康检查访问 127.0.0.1, 避免 Alpine 对 localhost 的 IPv6 解析差异。

5.4.2.1 前端运行阶段

```
FROM ${NGINX_IMAGE} AS runtime
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
HEALTHCHECK --interval=10s --timeout=3s \
  --start-period=5s --retries=3 \
  CMD wget -qO- http://127.0.0.1/ >/dev/null 2>&1 || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

最终镜像不包含 Node.js、源码和 node_modules, 减少体积与攻击面。前端资源文件名带内容哈希, index.html 不应长期缓存, assets 可设置较长缓存。

5.4.3 后端镜像构建

后端基于 Python 3.11 slim, 使用阿里云 Debian 与 PyPI 源, 安装依赖后复制应用和只读种子库。

```
ARG PYTHON_IMAGE=python:3.11-slim
ARG DEBIAN_MIRROR=http://mirrors.aliyun.com/debian
ARG PIP_INDEX_URL=https://mirrors.aliyun.com/pypi/simple

RUN apt-get update && apt-get install -y --no-install-recommends curl
RUN pip install -r requirements.txt --index-url "$PIP_INDEX_URL"
```

容器入口 scripts/docker_entrypoint.py 在 backend-data 为空时解压只读种子库, 之后启动 Uvicorn。若卷已有数据库, 重新构建镜像不会覆盖线上数据。

```
DATABASE_URL=sqlite:///./data/studymate.db
PISTON_URL=http://piston-api:2000
```

注意事项: 真实 backend/.env 不 COPY 到镜像, 只通过 env-file 在运行时注入。

5.4.3.1 后端运行阶段

```
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1
COPY app ./app
COPY scripts ./scripts
COPY resources ./resources
COPY run.py .
RUN test -s resources/seed/studymate.db.gz
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s \
    --start-period=20s --retries=5 \
    CMD curl -fsS http://localhost:8000/api/ping || exit 1
CMD ["python", "scripts/docker_entrypoint.py"]
```

入口脚本只在 /app/data/studymate.db 不存在时解压并播种数据库。线上卷已存在时, 即使镜像内种子库更新, 也不会隐式覆盖真实运行数据。

5.4.4 Caddy 与前端 Nginx

Caddyfile 使用 SITE-ADDRESS 作为站点地址，开启压缩、自动 HTTPS 和 SSE 低延迟转发。

```
{${SITE_ADDRESS:http://localhost}} {  
  encode zstd gzip  
  reverse_proxy frontend:80 {  
    flush_interval -1  
  }  
  header {  
    -Server  
    X-Content-Type-Options nosniff  
    Referrer-Policy strict-origin-when-cross-origin  
  }  
}
```

前端 Nginx 将 /api/ 转发至 backend:8000，关闭 SSE 缓冲并设置较长超时；其他路径使用 try_files 回退 index.html。

```
location /api/ {  
  proxy_pass http://backend:8000/api/;  
  proxy_http_version 1.1;  
  proxy_set_header Host $host;  
  proxy_set_header X-Forwarded-Proto $scheme;  
  proxy_buffering off;  
  proxy_cache off;  
  proxy_read_timeout 600s;  
  proxy_send_timeout 600s;  
  chunked_transfer_encoding on;  
}  
location / {  
  try_files $uri $uri/ /index.html;  
}
```

5.4.5 数据持久化

命名卷	容器挂载点	保存内容
studymate-backend-data	/app/data	SQLite 与上传文件
studymate-piston-data	/piston	Python/GCC runtime
studymate-caddy-data	/data	HTTPS 证书与 ACME 账户
studymate-caddy-config	/config	Caddy 运行配置

```
docker volume ls --format '{{.Name}}' \
| grep '^studymate_'
```

首次启动时 backend-data 从镜像种子库播种；后续更新只替换镜像和容器。Piston 与证书同样跨容器重建持久化。

```
docker volume inspect studymate_backend_data
docker volume inspect studymate_piston_data
docker volume inspect studymate_caddy_data
```

检查 Mountpoint、CreatedAt 和 Labels 可确认卷属于当前 Compose 项目。卷内文件由容器用户管理，不需要在宿主机执行 chmod 777，也不要直接进入 /var/lib/docker 修改数据。

持久化原则：如确需重置演示数据库，应先备份线上数据，再明确执行数据迁移；不要把 down -v 当作更新手段。

Compose 的 depends-on 与 healthcheck 确保：后端健康后启动前端，前端健康后启动 Caddy。

5.4.6 Piston 资源限制

限制项	配置值	原因
CPU	2.0 核	给后端和网关保留计算资源
内存	2 GB	防止无限申请内存拖垮宿主机
PID	256	限制进程/线程风暴
并发任务	2	默认 64 对 7GB 服务器过高
单任务进程	32	限制 fork 类代码
运行/编译超时	10s / 15s	避免长时间占用

```
docker inspect studymate-piston \
  --format 'memory={{.HostConfig.Memory}} cpus={{.HostConfig.NanoCpus}} pids={{.HostConfig.PidsLimit}}'
```

```
piston-api:
  privileged: true
  mem_limit: 2g
  cpus: 2.0
  pids_limit: 256
  environment:
    PISTON_MAX_CONCURRENT_JOBS: 2
    PISTON_MAX_PROCESS_COUNT: 32
    PISTON_RUN_TIMEOUT: 10000
    PISTON_COMPILE_TIMEOUT: 15000
```

这些是 Piston 容器整体硬上限。普通算法题、Python 和 C++17 编译运行已通过实测；如比赛需要较重任务，可按服务器余量谨慎提高，不建议取消。

5.5 一键启动与更新

5.5.1 首次启动

```
ssh studymate-server
cd ~/studymate
bash scripts/deploy.sh
```

脚本执行期间会显示镜像构建、容器创建、健康检查和 runtime 校验日志。首次构建完成后应看到四个核心容器。

容器	预期状态	公网端口
studymate-backend	Up (healthy)	无
studymate-frontend	Up (healthy)	无
studymate-piston	Up	无
studymate-caddy	Up	80、443

首次部署已在目标服务器实际完成；服务器自主构建前后端、Caddy 证书签发和 Piston runtime 均通过。

```
curl -fsS http://127.0.0.1:8000/api/ping
curl -fsSI http://127.0.0.1:5173/
curl -fsSI https://matropic.cn/
curl -fsS http://127.0.0.1:2000/api/v2/runtimes
```

四条命令分别验证后端健康、前端静态站点、公网 HTTPS 和代码运行时。任意一项失败时，应先查看对应容器日志，不要在未定位原因时反复删除容器或数据卷。

5.5.2 服务状态与日志

```
cd ~/studymate
bash scripts/deploy.sh status
bash scripts/deploy.sh logs
```

如只需要查看最近日志而不持续跟随：

```
docker compose --env-file .deploy.env logs \
--tail=200 backend frontend caddy piston-api
```

检查对象	成功标志
backend	health=healthy, /api/ping 200
frontend	health=healthy, 首页 200
caddy	certificate obtained successfully
piston	runtimes 包含 Python 与 C++

Docker 使用 local 日志驱动，单文件最大 100MB、最多 5 个，避免长期运行填满磁盘。

```
docker logs --tail=100 studymate-backend
docker logs --tail=100 studymate-frontend
docker logs --tail=100 studymate-caddy
docker logs --tail=100 studymate-piston
```

排查启动问题时先读取最近 100~200 行；复现 AI 流式回答或代码运行问题时再使用 logs -f 实时跟踪，完成后按 Ctrl+C 退出日志，不会停止容器。

5.5.3 仅更新前端

只修改 React 页面、样式或前端组件时，无需重建后端和 Piston。开发机同步前端：

```
rsync -az --delete --progress \  
  --exclude 'node_modules/' \  
  --exclude 'dist/' \  
  studymate/frontend/ \  
  studymate-server:~/studymate/frontend/
```

服务器仅重建 frontend：

```
ssh studymate-server \  
'cd ~/studymate && \  
  docker compose --env-file .deploy.env \  
  up -d --build frontend'
```

更新过程不会修改 backend-data。Caddy 保持运行，通常只有数秒静态资源切换时间。浏览器若缓存旧资源，可使用 Ctrl+F5。

注意事项：如同时修改 docker-compose.yml、Caddyfile 或后端接口，应走完整更新流程。

5.5.3.1 前端更新验收

```
docker compose --env-file .deploy.env ps frontend  
docker logs --tail=80 studymate-frontend  
curl -fsSI http://127.0.0.1:5173/  
curl -fsSI https://matropic.cn/  
  
# 查看本次构建生成的哈希资源  
docker exec studymate-frontend \  
  find /usr/share/nginx/html/assets -maxdepth 1 -type f | sort | tail
```

更新后先验证容器 healthy，再从公网检查首页。若浏览器仍显示旧内容，应比较 index.html 引用的哈希文件，而不是清理数据库或重启后端。

5.5.4 完整更新与回滚

```
rsync -az --delete --progress \
--exclude '.git/' \
--exclude 'frontend/node_modules/' \
--exclude 'backend/.venv/' \
--exclude 'backend/.env' \
--exclude 'backend/backups/' \
--exclude 'backend/studymate.db' \
--exclude 'backups/' \
--exclude '.deploy.env' \
--exclude '*.log' \
./studymate/ studymate-server:~/studymate/

ssh studymate-server \
'cd ~/studymate && bash scripts/deploy.sh'
```

更新前建议记录 Git 提交号和备份数据库。回滚时恢复旧代码或旧镜像后重新执行 deploy.sh，命名卷仍保留。

```
git rev-parse --short HEAD
docker image ls 'studymate-*' --digests
```

注意事项：不要把服务器 backend/.env、.deploy.env 或备份目录通过 --delete 同步掉；必须保留对应 exclude。

5.5.4.1 回滚记录

记录项	示例	回滚用途
代码版本	Git short SHA	恢复对应源码
镜像摘要	RepoDigest/Image ID	确认实际运行镜像
数据库备份	日期时间.db	恢复业务数据
配置摘要	.deploy.env 变量名	恢复域名/profile
验收结果	登录/AI/代码	判断回归范围

回滚代码与回滚数据是两件不同的操作。一般代码回滚只替换镜像并保留当前卷；只有确认数据库迁移或数据损坏时，才在停机和二次备份后恢复旧数据库。

5.6 权限与安全加固

deploy 用户使用 SSH 密钥登录，不在文档中保存密码或私钥。

backend/.env 与 .deploy.env 权限为 600。

Docker 组等同较高系统权限，只授予可信维护人员。

FastAPI、前端排障端口和 Piston 仅监听 127.0.0.1。

会话 Cookie 在线上启用 Secure、HttpOnly、SameSite=Lax。

Caddy 移除 Server 头并设置 nosniff、Referrer-Policy。

测试账号仅用于竞赛演示，正式运营前必须更换密码。

Piston 保持容器资源上限，不把 2000 端口暴露公网。

```
chmod 600 ~/studymate/.deploy.env
chmod 600 ~/studymate/backend/.env
ss -lnt | grep -E ':(80|443|2000|5173|8000)'
```

预期只有 80/443 监听公网地址，其他端口显示 127.0.0.1。

安全对象	控制措施
SSH	密钥登录、限制来源 IP、普通 deploy 用户
环境变量	权限 600、Rsync 排除、运行时注入
Web	HTTPS、Secure Cookie、安全响应头
代码沙箱	仅本机端口、容器整体资源与超时限制
数据	命名卷持久化、更新前备份、禁止 down -v

6. SSL 证书配置

6.1 Caddy 自动 HTTPS

本项目不使用自签名证书，也不需要手工安装 Certbot。Caddy 根据 SITE-ADDRESS 自动注册 ACME 账户、完成 HTTP-01 验证并保存证书。

前提条件	要求
DNS	matropic.cn A 记录指向 121.40.64.199
安全组	80、443 对公网开放
UFW	允许 80/tcp、443/tcp
Caddy	容器正常运行，SITE-ADDRESS=matropic.cn

```
docker logs --tail=120 studymate-caddy
```

成功日志包含 validations succeeded、certificate obtained successfully。证书存放在 caddy-data 卷中。

```
getent ahostsv4 matropic.cn
sudo ufw status | grep -E '80|443'
docker port studymate-caddy
```

申请证书前，域名解析结果必须与服务器公网 IP 一致，Caddy 容器必须实际映射 80 和 443。若 DNS 刚修改，可等待 TTL 到期后再次验证。

注意事项：如果 DNS 尚未生效或 80 端口被安全组阻断，Caddy 会自动重试；不要改用“继续访问不安全站点”的方式绕过。

6.2 证书验证与续期

```
curl -I http://matropic.cn
curl -I https://matropic.cn
openssl s_client \
  -connect matropic.cn:443 \
  -servername matropic.cn </dev/null
```

验证项	实际结果
HTTP	308 Permanent Redirect
HTTPS	HTTP/2 200
证书颁发	Let's Encrypt
域名	matropic.cn
自动续期	由 Caddy 管理

Caddy 会根据证书续期窗口自动更新，无需 cron。应保留 `caddy_data` 和 `caddy_config`，并定期检查日志。

```
docker compose --env-file .deploy.env \
  logs --tail=100 caddy
```

```
# 预期响应摘要
HTTP/1.1 308 Permanent Redirect
Location: https://matropic.cn/

HTTP/2 200
issuer=Let's Encrypt
```

验收时还应在手机流量或其他外部网络中打开站点，排除仅服务器本机可访问的假阳性；浏览器证书详情中的域名、有效期和颁发者均应正常。

7. 验证部署

7.1 检查服务状态

```
cd ~/studymate
docker compose --env-file .deploy.env ps
docker stats --no-stream
```

服务	实际状态	说明
backend	healthy	FastAPI 健康检查通过
frontend	healthy	Nginx 静态站点正常
caddy	running	80/443 正常
piston-api	running	runtime 已加载

7.1.1 检查持久化

```
docker volume ls --format '{{.Name}}' \
| grep '^studymate_'
```

backend、Piston、Caddy 数据卷均应存在。

```
docker inspect studymate-backend \
--format '{{.State.Status}} / {{.State.Health.Status}}'
docker inspect studymate-frontend \
--format '{{.State.Status}} / {{.State.Health.Status}}'
```

若状态为 starting，可等待健康检查的 start-period；若连续变为 unhealthy，则查看容器日志和健康检查输出，而不是只依赖 Compose 的 Up 状态。

7.2 检查端口与公网访问

```
ss -lntp | grep -E \
':(22|80|443|2000|5173|8000)'
```

```
curl -I https://matropic.cn
curl https://matropic.cn/api/ping
```

端口	监听地址	验收
80	0.0.0.0 / ::	公网 HTTP 跳转
443	0.0.0.0 / ::	公网 HTTPS
5173	127.0.0.1	仅本机
8000	127.0.0.1	仅本机
2000	127.0.0.1	仅本机

公网首页与 /api/ping 均已返回 200，域名访问不出现浏览器证书警告。

```
sudo ufw status numbered
curl -I http://matropic.cn
curl -I https://matropic.cn
curl -sS https://matropic.cn/api/ping | python3 -m json.tool
```

公网测试应从服务器以外的网络执行。5173、8000 和 2000 即使绑定宿主机回环，也不应在阿里云安全组中添加放行规则。

7.3 核心功能验收

功能	验收方法	结果
登录	评委账号登录并访问 /api/auth/me	通过
Cookie	检查 Set-Cookie 的 Secure/HttpOnly	通过
课程	切换五门课程并读取知识库	通过
AI 助教	SSE 流式回答指定短语	真实模型、done 事件正常
外部资源	高相关课程与视频、可信阅读直链或稳定搜索兜底	通过
备案页脚	公网 JS 与页面包含备案号	通过

```
curl https://matropic.cn/api/ping
# 返回 status=ok、service=studymate-backend
```

登录响应使用安全 Cookie; SSE 返回 meta、delta、done 事件, meta 中 mock=false。阅读解析只返回经过标题、HTTPS、主机和路径校验的 arXiv、DOI、豆瓣图书、CSDN 或掘金详情页; 未命中时由前端保留搜索入口。

页面底部显示“豫ICP备2026028221号”, 并链接 <https://beian.miit.gov.cn/>。

- 使用 admin、judge 和普通学生三类账号分别登录, 确认角色权限边界。
- 在五门课程之间切换, 确知识库、会话和生成结果不会跨课程串用。
- 提交 AI 问题后观察内容逐段到达, 而不是长时间等待后一次性返回。
- 上传文本或 PDF, 确认文件大小、类型校验与解析提示符合预期。
- 刷新浏览器后会话仍有效, 退出登录后旧 Cookie 不能继续访问受保护接口。

7.4 数据与代码运行验收

项目	测试内容	实际结果
文件上传	上传 2MB 文本文件	HTTP 200, 内容截断策略正常
Python	<code>print(6 * 7)</code>	<code>stdout=42, mock=false</code>
C++17	编译输出 42	编译/运行退出码 0
用户数据	指定账号集合	19 个且完全一致
课程数据	课程与知识块	5 门、938 块
SQLite	<code>integrity-check/foreign-key-check</code>	<code>ok / 0</code>

当前本地质量基线还包括 23 项后端自动化测试，以及建议 2、建议 3、建议 4 三组前端隔离回归；覆盖画像与报告安全写回、未作答兼容、RAG 相对匹配度、外部资源相关性、可信阅读直链和就业雷达更新。

```
curl http://127.0.0.1:2000/api/v2/runtimes

docker exec -i studymate-backend python <<'PY'
import sqlite3
c = sqlite3.connect('/app/data/studymate.db')
print(c.execute('pragma integrity_check').fetchone())
PY
```

7.4.1 代码沙箱输入边界

项目	后端限制
源码	最大 50KB
标准输入	最大 10KB
命令行参数	最多 16 个
运行超时	10 秒
编译超时	15 秒
并发	Piston 容器整体最多 2 个任务

```
# Python 冒烟测试
curl -sS https://matropic.cn/api/run \
-H 'Content-Type: application/json' \
-d '{"language":"python","source":"print(6*7)}'
```

7.5 运行监控与验收结论

验收维度	结论
可访问性	HTTP 自动跳 HTTPS, HTTPS 200
容器健康	前后端 healthy, Caddy/Piston running
数据完整	19 用户、5 课程、938 知识块, 完整性正常
AI 能力	真实模型流式回复完成
在线编程	Python 与 C++ 真实执行成功
安全边界	仅 22/80/443 公网开放, Cookie Secure
资源余量	59GB 系统盘, 验收时约 46GB 可用

结论: StudyMate 已在 Ubuntu 22.04 服务器完成可重复 Docker 部署, 满足竞赛公网展示、评委账号访问、核心业务演示和后续前端快速更新要求。

验收记录应与具体版本绑定, 至少保存部署日期、Git 提交号、镜像摘要、数据库备份文件名和测试人员。出现回归时可据此快速判断是代码、配置、数据还是外部模型服务发生变化。

- 1. 上线前: 备份数据库并记录当前容器、镜像状态。
- 2. 上线后: 检查四个服务、端口、证书和数据卷。
- 3. 业务侧: 复测登录、课程、AI、上传、代码运行与备案信息。
- 4. 异常时: 保留日志与失败请求, 不执行 `down -v` 或手工覆盖数据库。

验收基线: 每次重大更新后至少复测首页、登录、`/api/ping`、AI SSE、上传、Python/C++ 和数据库完整性。

8. 常见问题排查

8.1 容器、反向代理与镜像问题

症状	排查与处理
docker: command not found	确认初始化脚本以 root 执行完成；检查安装日志。
permission denied docker.sock	退出 SSH 后重新登录，确认用户属于 docker 组。
镜像拉取超时	检查 daemon.json、重启 Docker，或在 .deploy.env 使用显式代理镜像。
502 Bad Gateway	检查 backend health 与 frontend 日志，确认 backend:8000 可达。
前端 unhealthy	检查 Nginx 配置、构建产物和 healthcheck 的 127.0.0.1。

```
bash scripts/deploy.sh status
docker compose --env-file .deploy.env \
  logs --tail=200 backend frontend
```

8.1.1 分层定位顺序

```
# 1. 容器与健康状态
docker compose --env-file .deploy.env ps -a
# 2. 后端本机健康
curl -v http://127.0.0.1:8000/api/ping
# 3. 前端容器到后端
docker exec studymate-frontend \
  wget -qO- http://backend:8000/api/ping
# 4. 前端本机入口
curl -I http://127.0.0.1:5173/
# 5. Caddy 公网入口
curl -Iv https://matropic.cn/
```

按照后端、Docker 内网、前端、Caddy 的顺序逐层验证，可快速确定 502 出现在请求链路的哪一段。不要只看浏览器报错就直接重装 Docker。

8.2 HTTPS、Piston 与前端缓存问题

症状	排查与处理
证书申请失败	确认 DNS A 记录、安全组 80/443、UFW 与 Caddy 日志。
GHCR 拉取停滞	执行 <code>import-piston-image.sh</code> ，必要时从本机 <code>docker save</code> 传输。
runtime 为空	运行 <code>init-piston.sh</code> ，并检查 <code>piston-data</code> 卷。
代码运行 mock	检查 backend 到 <code>piston-api:2000</code> 的 Docker 内网连接。
前端仍是旧版	重建 frontend，检查新哈希资源，浏览器 <code>Ctrl+F5</code> 。
上传 413	检查前端 <code>Nginx client-max-body-size</code> 与后端 10MB 规则。

```
docker logs --tail=200 studymate-caddy
curl http://127.0.0.1:2000/api/v2/runtimes
docker volume inspect studymate_piston_data
```

8.2.1 证书与沙箱专项检查

```
getent ahostsv4 matropic.cn
docker logs --tail=200 studymate-caddy
openssl s_client -connect matropic.cn:443 \
  -servername matropic.cn </dev/null 2>/dev/null \
  | openssl x509 -noout -issuer -dates

docker inspect studymate-piston --format '{{.State.Status}}'
curl -sS http://127.0.0.1:2000/api/v2/runtimes \
  | python3 -m json.tool
```

沙箱边界：Piston 使用 `privileged` 提供 `isolate` 所需 `namespace` 能力；必须同时保持 127.0.0.1 端口绑定、容器整体资源上限和输入/超时限制。

9. 日常运维

9.1 查看日志与服务管理

```
cd ~/studymate
bash scripts/deploy.sh status
bash scripts/deploy.sh logs

docker compose --env-file .deploy.env restart backend
docker compose --env-file .deploy.env restart frontend
docker compose --env-file .deploy.env restart caddy
docker stats --no-stream
df -h
```

需要停止整个应用时使用 `bash scripts/deploy.sh down`; 恢复使用 `deploy.sh`。该 `down` 不删除卷。

日志对象	命令
后端	<code>docker logs --tail=200 studymate-backend</code>
前端	<code>docker logs --tail=200 studymate-frontend</code>
Caddy	<code>docker logs --tail=200 studymate-caddy</code>
Piston	<code>docker logs --tail=200 studymate-piston</code>

9.1.1 自动恢复与重启次数

```
systemctl is-enabled docker
systemctl is-active docker
docker inspect studymate-backend \
  --format 'restart={{.HostConfig.RestartPolicy.Name}} count={{.RestartCount}}'
docker inspect studymate-frontend \
  --format 'restart={{.HostConfig.RestartPolicy.Name}} count={{.RestartCount}}'
docker inspect studymate-caddy \
  --format 'restart={{.HostConfig.RestartPolicy.Name}} count={{.RestartCount}}'
```

核心容器使用 `unless-stopped`，服务器重启后会随 Docker 恢复。`RestartCount` 持续增加通常表示应用崩溃，应先查看日志和资源占用，而不是依赖无限重启掩盖问题。

9.2 数据备份与恢复

使用 SQLite 在线备份 API 可在不中断写入的情况下生成一致副本:

```
mkdir -p ~/studymate-backups
docker exec studymate-backend python -c '
import sqlite3
s=sqlite3.connect("/app/data/studymate.db")
d=sqlite3.connect("/app/data/studymate-backup.db")
s.backup(d); d.close(); s.close()'
docker cp studymate-backend:/app/data/studymate-backup.db \
~/studymate-backups/studymate-$(date +%F_%H%M%S).db
docker exec studymate-backend rm -f /app/data/studymate-backup.db
```

9.3 性能与资源管理

定期检查磁盘与 Docker 镜像缓存。

Piston 容器整体保持 2CPU、2GB 和 2 并发上限。

长时间运行后检查日志与容器重启次数。

扩大并发前先压测 AI、SQLite 与代码沙箱。

注意事项: 恢复数据库前必须先备份当前文件, 并停止 backend; 恢复后再次执行完整性检查。

9.2.1 恢复与备份轮换

```
# 恢复前: 再次备份当前数据库并停止后端
docker compose --env-file .deploy.env stop backend
docker run --rm \
-v studymate_backend_data:/data \
-v "$HOME/studymate-backups:/backup:ro" \
alpine:3.20 sh -c \
'cp /backup/指定备份.db /data/studymate.db && rm -f /data/studymate.db-wal /data/studymate.db-shm'
docker compose --env-file .deploy.env start backend
curl -fsS http://127.0.0.1:8000/api/ping

# 只保留人工确认后的历史备份; 删除前先列出
find ~/studymate-backups -type f -name '*.db' -printf '%TY-%Tm-%Td %p'
| sort
```

恢复后需要复核用户、课程、知识块和外键完整性。备份应至少保留一份在服务器之外, 避免系统盘故障或误操作同时损坏线上卷与本机备份目录。

10. 附录

10.1 文件说明与部署状态

```
studymate/
├── backend/           # FastAPI、本地运行库与压缩种子资源
├── frontend/         # React、Nginx 配置
├── scripts/          # 初始化、部署、数据清理
├── docs/             # 部署说明
├── docker-compose.yml # 服务、网络、卷、限制
├── Caddyfile         # HTTPS 公网入口
├── .deploy.env.example # 生产 Compose 示例
└── README.md
```

生产项	当前状态
公网地址	https://matropic.cn
ICP备案	豫ICP备2026028221号
核心容器	4 个，运行正常
HTTPS	Let's Encrypt，自动续期
数据库	19 用户、5 课程、938 知识块
代码运行	Python/C/C++ runtime 已安装

10.2 测试网址与账号



图 10-1 StudyMate 公网访问二维码

公网访问地址: <https://matropic.cn>

管理员账号: `admin@studymate.com` 密码: `admin123456`

评委账号: `judge01@studymate.com` 至 `judge10@studymate.com` 统一密码: `judge123456`

学生账号: `sunjiayu`、`baixinyue`、`yuanshcong`、`chenzhuo`、`lijiayi`、`zhouxiang`、`tianyixin`、`liufei`，
邮箱域均为 `@studymate.com`，统一密码: `user123456`。

测试账号声明：以上账号仅用于竞赛测试环境。文档对外公开或转为正式运营前，应删除密码或执行统一轮换。

浏览器应直接显示可信 HTTPS，不需要点击“继续访问不安全站点”。

10.3 项目代码与交付清单

项目代码随竞赛作品压缩包提交，服务器部署目录为 /home/deploy/studymate。部署脚本已包含国内镜像源、Docker Compose 启动和 Piston 初始化逻辑。

交付文件	用途
docker-compose.yml	生产容器编排
Caddyfile	公网 HTTPS 与反向代理
scripts/bootstrap-ubuntu.sh	服务器一次性初始化
scripts/deploy.sh	一键构建、启动、状态与日志
scripts/import-piston-image.sh	GHCR 受限时导入镜像
scripts/init-piston.sh	幂等安装并校验 runtime
scripts/build-seed-db.py	生成脱敏压缩部署种子
scripts/sanitize-demo-db.py	演示账号数据清理

最终验收清单

- 域名与 HTTPS 正常
- 四个核心容器运行
- 19 个指定账号准确
- 5 门课程与 938 知识块完整
- AI SSE 为真实模型
- Python/C++ 运行通过
- 备案号可见
- 备份与更新命令可执行

```
# 交付压缩包和公开仓库必须排除
.git/
frontend/node_modules/
frontend/dist/
backend/.venv/
backend/.env
.deploy.env
backend/backups/
*.log
SSH 私钥与真实 API Key
```

提交前检查：作者姓名、代码仓库或网盘链接应在正式提交前补齐；公开版本若不需要直接展示测试密码，可将账号密码移至单独的《评委访问说明》。